Лабораторная работа 7 - Объектно-ориентированное программирование в Python

Подготовил студент группы 523 Чех Степан

Задание 1

Создать класс ThreeStates и определить у него один атрибут state и метод hasstate. Атрибут реализовать как приватный.
В рамках метода реализовать переключение состояний: первое, второе, третье.
Смена состояния происходит в момент вызова метода hasstate и сопровождается выводом на экран соответствующего сообщения.
Переключение между состояниями должно осуществляться только в указанном порядке (первое, второе, третье, первое, ...).
Проверить работу примера, создав экземпляр и вызвав описанный метод.

```
class ThreeStates():
  # Приватный атрибут класса: кортеж всех допустимых состояний
    __states = (1,2,3)
    # текущее состояние
    __state = 1
    # При создании объекта сразу проверяем валидность и пытаемся применить переданное состояние
    def __init__(self, state):
        self.hasstate(state)

    # Условие смены состояния состоит из двух частей:
        # 1. Новое состояние должно входить в список допустимых (1, 2 или 3)
        # 2. Новое состояние должно строго следовать за текущим по формуле (текущее + 1) % 3
    def hasstate(self, state):
        if state in self.__states and (self.__state + 1)%3 == state:
            self.__state = state
            print(f"Состояние поменялось на {self.__state}")
        else:
            print(f"{state} не валидно, сейчас состояние {self.__state}")

mkpp = ThreeStates(3)
mkpp.hasstate(2)
mkpp.hasstate(1)
```

```
3 не валидно, сейчас состояние 1
Состояние поменялось на 2
1 не валидно, сейчас состояние 2
```

Задание 2

Реализовать класс Volume, в котором определить атрибуты: length (длина), width (ширина) и height (высота).
Значения данных атрибутов должны передаваться при создании экземпляра класса.
Атрибуты сделать защищенными. Определить метод расчета, использовав формулу: длина * ширина * высота.
Проверить работу метода.

```
class Volume:
    # Конструктор класса: вызывается автоматически при создании объекта
    # Параметры: lenght (длина), width (ширина), height (высота)
    def __init__(self, lenght, width, height):
        # Сохраняем переданные значения как атрибуты экземпляра
        self._lenght = lenght
        self._width = width
        self._height = height

    # Метод вычисления объёма прямоугольного параллелепипеда
    def getvolume(self):
        return self._height * self._width * self._lenght

# Создаём объект класса с размерами 6, 2 и 3
fig_1 = Volume(6, 2, 3)
# Вызываем метод расчёта.
fig_1.getvolume()
```

```
36
```

Задание 3

Реализовать базовый класс Employee, в котором определить атрибуты: name, patronymic, surname, salary.
Последний атрибут должен быть защищенным и ссылаться на словарь, содержащий элементы: жалование и бонус, например, {"wage": wage, "bonus": bonus}.
Создать класс Salary на базе класса Employee. В классе Salary реализовать методы получения полного имени сотрудника (get_full_name) и дохода с учетом премии (get_total_income).
Создать экземпляры класса Salary, передать данные, проверить значения атрибутов, вызвать методы экземпляров.

```
class Employee:
    # Конструктор родительского класса
    def __init__(self, name, patronymic, surname, wage, bonus):
        # Публичные атрибуты: доступны напрямую извне
        self.name = name
        self.patronymic = patronymic
        self.surname = surname
         # Защищённый атрибут: словарь с компонентами дохода
        self._salary = {'wage': wage, 'bonus': bonus }

class Salary(Employee):
    # Наследует __init__ и атрибуты от Employee, так как своего конструктора нет
    def get_full_name(self):
        return f"{self.surname} {self.name} {self.patronymic}"

    def get_total_income(self):
        return self._salary['wage'] + self._salary['bonus']

sal_1 = Salary("Степан", "Владимирович", "Чех", 500, 500000)

print(sal_1.name)
print(sal_1.patronymic)
print(sal_1.surname)
print(sal_1._salary)

print(sal_1.get_full_name())
print(sal_1.get_total_income())
```

```
Степан
Владимирович
Чех
{'wage': 500, 'bonus': 500000}
Чех Степан Владимирович
500500
```

Задание 4

Реализуйте базовый класс Airplane. У данного класса должны быть следующие атрибуты: speed, color, name, is_jet (булево).
А также методы: go, stop, direction, которые должны сообщать, что самолет летит, не летит, повернул в полете.
Опишите несколько дочерних классов самолетов: FastAirplane, Biplane, ArmyAirplane.
Добавьте в базовый класс метод show_speed, который должен показывать текущую скорость самолета.
Для дочерних классов переопределите метод show_speed.
При значении скорости свыше 1300 (FastAirplane) должно выводиться сообщение о сверхзвуковой скорости.
Создайте экземпляры классов, передайте значения атрибутов.
Выполните доступ к атрибутам, выведите результат.
Выполните вызов методов и также покажите результат.

```
class Airplane:
    def __init__(self, speed, color, name, is_jet):
        self.speed = speed
        self.color = color
        self.name = name
        self.is_jet = is_jet

    def go(self):
        print("Самолёт летит")
        if self.speed == 0:
            self.speed += 100

    def stop(self):
        print("Самолет не летит")
        self.speed = 0

    def direction(self):
        print("Самолёт повернул")
```

```
    def show_speed(self):
        pass


class FastAirplane(Airplane):
    def show_speed(self):
        if self.speed > 1300:
            print(f"Сверхзвуковая скорость - {self.speed} км\ч")
        else:
            print(f"Маловато - всего {self.speed} км\ч")

class Biplane(Airplane):
    def show_speed(self):
        print(f"Ползем на скоростях {self.speed} км\ч")

class ArmyAirplane(Airplane):
    def show_speed(self):
        print(f"Летим на скорости {self.speed} км\ч")


basic_plane = Airplane(0, "коричневый", "победа", True)
basic_plane.go()
basic_plane.stop()
basic_plane.direction()
basic_plane.show_speed()
print()

fast_plane = FastAirplane(1500, "белый", "j7", True)
fast_plane.go()
fast_plane.direction()
fast_plane.show_speed()
fast_plane.speed = 1000
fast_plane.show_speed()
fast_plane.stop()
print()

bi_plane = Biplane(15, "жёлтый", "Самолёт", True)
bi_plane.go()
bi_plane.direction()
bi_plane.show_speed()
print()

army_plane = ArmyAirplane(120, "зелёный", "воин", True)
army_plane.go()
army_plane.direction()
army_plane.show_speed()
```

```
Самолёт летит
Самолет не летит
Самолёт повернул

Самолёт летит
Самолёт повернул
Сверхзвуковая скорость - 1500 км\ч
Маловато - всего 1000 км\ч
Самолет не летит

Самолёт летит
Самолёт повернул
Ползем на скоростях 15 км\ч

Самолёт летит
Самолёт повернул
Летим на скорости 120 км\ч
```

Задание 5

Реализовать класс MathOperations.
Определить в нем атрибуты first_num, second_num и метод calc.
Метод выводит сообщение “Запуск операции.”
Создать три дочерних класса my_sum (сложение), my_sub (вычитание), my_mult (умножение).
В каждом из классов реализовать переопределение метода calc.
Для каждого из классов метод должен выполнять математическую операцию, соответствующую названию класса.
Создать экземпляры классов и проверить работу методов.

```
class MathOperations:
    def __init__(self, first_num, second_num):
        self.first_num = first_num
```

```
        self.second_num = second_num

    def calc(self):
        print("Запускаю вычисления...")

class MySum(MathOperations):
    def calc(self):
        print(f"Результат равен {self.first_num + self.second_num}")
        return self.first_num + self.second_num

class MySub(MathOperations):
    def calc(self):
        print(f"Результат равен {self.first_num / self.second_num}")
        return self.first_num / self.second_num

class MyMult(MathOperations):
    def calc(self):
        print(f"Результат равен {self.first_num * self.second_num}")
        return self.first_num * self.second_num

base = MathOperations(10, 9)
base.calc()

summ = MySum(97412, 78432)
summ.calc()

sub = MySub(1874, 4)
sub.calc()

mult = MyMult(9813, 612)
mult.calc()
```

```
Запускаю вычисления...
Результат равен 175844
Результат равен 468.5
Результат равен 6005556
6005556
```